arXiv:2105.01601v4 [cs.CV] 11 Jun 2021

MLP-Mixer: An all-MLP Architecture for Vision

Ilya Tolstikhin*, **Neil Houlsby***, **Alexander Kolesnikov***, **Lucas Beyer***,
Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner,
Daniel Keysers, Jakob Uszkoreit, Mario Lucic, Alexey Dosovitskiy
*equal contribution
Google Research, Brain Team
{tolstikhin, neilhoulsby, akolesnikov, lbeyer, xzhai, unterthiner, jessicayung†, andstein, keysers, usz, lucic, adosovitskiy}@google.com
†work done during Google AI Residency

Abstract

Convolutional Neural Networks (CNNs) are the go-to model for computer vision. Recently, attention-based networks, such as the Vision Transformer, have also become popular. In this paper we show that while convolutions and attention are both sufficient for good performance, neither of them are necessary. We present *MLP-Mixer*, an architecture based exclusively on multi-layer perceptrons (MLPs). MLP-Mixer contains two types of layers: one with MLPs applied independently to image patches (i.e. "mixing" the per-location features), and one with MLPs applied across patches (i.e. "mixing" spatial information). When trained on large datasets, or with modern regularization schemes, MLP-Mixer attains competitive scores on image classification benchmarks, with pre-training and inference cost comparable to state-of-the-art models. We hope that these results spark further research beyond the realms of well established CNNs and Transformers.[1]

1 Introduction

As the history of computer vision demonstrates, the availability of larger datasets coupled with increased computational capacity often leads to a paradigm shift. While Convolutional Neural Networks (CNNs) have been the de-facto standard for computer vision, recently Vision Transformers [14] (ViT), an alternative based on self-attention layers, attained state-of-the-art performance. ViT continues the long-lasting trend of removing hand-crafted visual features and inductive biases from models and relies further on learning from raw data.

We propose the *MLP-Mixer* architecture (or "Mixer" for short), a competitive but conceptually and technically simple alternative, that does not use convolutions or self-attention. Instead, Mixer's architecture is based entirely on multi-layer perceptrons (MLPs) that are repeatedly applied across either spatial locations or feature channels. Mixer relies only on basic matrix multiplication routines, changes to data layout (reshapes and transpositions), and scalar nonlinearities.

Figure 1 depicts the macro-structure of Mixer. It accepts a sequence of linearly projected image patches (also referred to as *tokens*) shaped as a "patches $\times$ channels" table as an input, and maintains this dimensionality. Mixer makes use of two types of MLP layers: *channel-mixing MLPs* and *token-mixing MLPs*. The channel-mixing MLPs allow communication between different channels;

[1] MLP-Mixer code will be available at `https://github.com/google-research/vision_transformer`

Preprint. Under review.

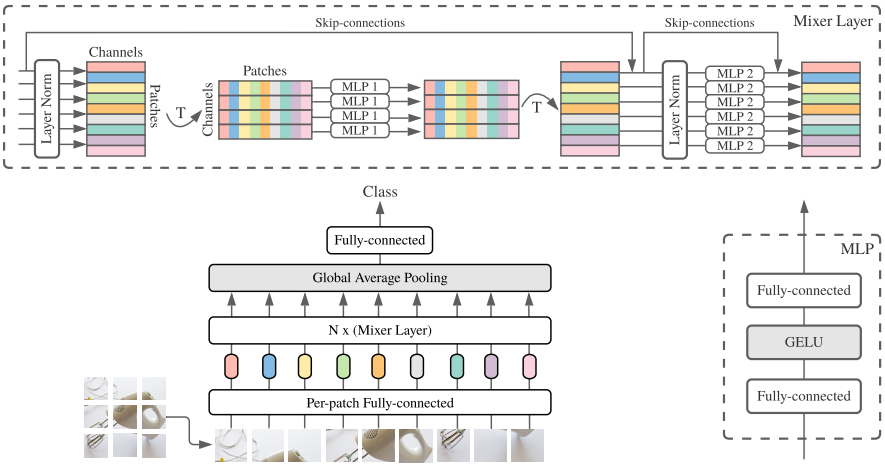

Figure 1: MLP-Mixer consists of per-patch linear embeddings, Mixer layers, and a classifier head. Mixer layers contain one token-mixing MLP and one channel-mixing MLP, each consisting of two fully-connected layers and a GELU nonlinearity. Other components include: skip-connections, dropout, and layer norm on the channels.

they operate on each token independently and take individual rows of the table as inputs. The token-mixing MLPs allow communication between different spatial locations (tokens); they operate on each channel independently and take individual columns of the table as inputs. These two types of layers are interleaved to enable interaction of both input dimensions.

In the extreme case, our architecture can be seen as a very special CNN, which uses 1×1 convolutions for *channel mixing*, and single-channel depth-wise convolutions of a full receptive field and parameter sharing for *token mixing*. However, the converse is not true as typical CNNs are not special cases of Mixer. Furthermore, a convolution is more complex than the plain matrix multiplication in MLPs as it requires an additional costly reduction to matrix multiplication and/or specialized implementation.

Despite its simplicity, Mixer attains competitive results. When pre-trained on large datasets (i.e., $\sim$100M images), it reaches near state-of-the-art performance, previously claimed by CNNs and Transformers, in terms of the accuracy/cost trade-off. This includes 87.94% top-1 validation accuracy on ILSVRC2012 "ImageNet" [13]. When pre-trained on data of more modest scale (i.e., $\sim$1–10M images), coupled with modern regularization techniques [49, 54], Mixer also achieves strong performance. However, similar to ViT, it falls slightly short of specialized CNN architectures.

2 Mixer Architecture

Modern deep vision architectures consist of layers that mix features (i) at a given spatial location, (ii) between different spatial locations, or both at once. In CNNs, (ii) is implemented with $N \times N$ convolutions (for $N > 1$) and pooling. Neurons in deeper layers have a larger receptive field [1, 28]. At the same time, 1×1 convolutions also perform (i), and larger kernels perform both (i) and (ii). In Vision Transformers and other attention-based architectures, self-attention layers allow both (i) and (ii) and the MLP-blocks perform (i). The idea behind the Mixer architecture is to clearly separate the per-location (*channel-mixing*) operations (i) and cross-location (*token-mixing*) operations (ii). Both operations are implemented with MLPs. Figure 1 summarizes the architecture.

Mixer takes as input a sequence of S non-overlapping image patches, each one projected to a desired *hidden dimension* C. This results in a two-dimensional real-valued input table, $\mathbf{X} \in \mathbb{R}^{S\times C}$. If the original input image has resolution (H, W), and each patch has resolution (P, P), then the number of patches is $S = HW/P^2$. All patches are linearly projected with the *same* projection matrix. Mixer consists of multiple layers of identical size, and each layer consists of two MLP blocks. The first one is the *token-mixing* MLP: it acts on columns of $\mathbf{X}$ (i.e. it is applied to a transposed input table $\mathbf{X}^\top$), maps $\mathbb{R}^S \mapsto \mathbb{R}^S$, and is shared across all columns. The second one is the *channel-mixing* MLP: it acts on rows of $\mathbf{X}$, maps $\mathbb{R}^C \mapsto \mathbb{R}^C$, and is shared across all rows. Each MLP block contains two

fully-connected layers and a nonlinearity applied independently to each row of its input data tensor. Mixer layers can be written as follows (omitting layer indices):

$$\mathbf{U}_{*,i} = \mathbf{X}_{*,i} + \mathbf{W}_2\, \sigma\big(\mathbf{W}_1\, \mathrm{LayerNorm}(\mathbf{X})_{*,i}\big), \quad \text{for } i = 1 \ldots C, \tag{1}$$
$$\mathbf{Y}_{j,*} = \mathbf{U}_{j,*} + \mathbf{W}_4\, \sigma\big(\mathbf{W}_3\, \mathrm{LayerNorm}(\mathbf{U})_{j,*}\big), \quad \text{for } j = 1 \ldots S.$$

Here σ is an element-wise nonlinearity (GELU [16]). D_S and D_C are tunable hidden widths in the token-mixing and channel-mixing MLPs, respectively. Note that D_S is selected independently of the number of input patches. Therefore, the computational complexity of the network is linear in the number of input patches, unlike ViT whose complexity is quadratic. Since D_C is independent of the patch size, the overall complexity is linear in the number of pixels in the image, as for a typical CNN.

As mentioned above, the *same* channel-mixing MLP (token-mixing MLP) is applied to every row (column) of $\mathbf{X}$. Tying the parameters of the channel-mixing MLPs (within each layer) is a natural choice—it provides positional invariance, a prominent feature of convolutions. However, tying parameters across channels is much less common. For example, separable convolutions [9, 40], used in some CNNs, apply convolutions to each channel independently of the other channels. However, in separable convolutions, a different convolutional kernel is applied to each channel unlike the token-mixing MLPs in Mixer that share the same kernel (of full receptive field) for all of the channels. The parameter tying prevents the architecture from growing too fast when increasing the hidden dimension C or the sequence length S and leads to significant memory savings. Surprisingly, this choice does not affect the empirical performance, see Supplementary A.1.

Each layer in Mixer (except for the initial patch projection layer) takes an input of the same size. This "isotropic" design is most similar to Transformers, or deep RNNs in other domains, that also use a fixed width. This is unlike most CNNs, which have a *pyramidal* structure: deeper layers have a lower resolution input, but more channels. Note that while these are the typical designs, other combinations exist, such as isotropic ResNets [38] and pyramidal ViTs [52].

Aside from the MLP layers, Mixer uses other standard architectural components: skip-connections [15] and layer normalization [2]. Unlike ViTs, Mixer does not use position embeddings because the token-mixing MLPs are sensitive to the order of the input tokens. Finally, Mixer uses a standard classification head with the global average pooling layer followed by a linear classifier. Overall, the architecture can be written compactly in JAX/Flax, the code is given in Supplementary E.

3 Experiments

We evaluate the performance of MLP-Mixer models, pre-trained with medium- to large-scale datasets, on a range of small and mid-sized downstream classification tasks. We are interested in three primary quantities: (1) Accuracy on the downstream task; (2) *Total* computational cost of pre-training, which is important when training the model from scratch on the upstream dataset; (3) Test-time throughput, which is important to the practitioner. Our goal is not to demonstrate state-of-the-art results, but to show that, remarkably, a simple MLP-based model is competitive with today's best convolutional and attention-based models.

Downstream tasks We use popular downstream tasks such as ILSVRC2012 "ImageNet" (1.3M training examples, 1k classes) with the original validation labels [13] and cleaned-up ReaL labels [5], CIFAR-10/100 (50k examples, 10/100 classes) [23], Oxford-IIIT Pets (3.7k examples, 36 classes) [32], and Oxford Flowers-102 (2k examples, 102 classes) [31]. We also use the Visual Task Adaptation Benchmark (VTAB-1k), which consists of 19 diverse datasets, each with 1k training examples [58].

Pre-training We follow the standard transfer learning setup: pre-training followed by fine-tuning on the downstream tasks. We pre-train our models on two public datasets: ILSVRC2021 ImageNet, and ImageNet-21k, a superset of ILSVRC2012 that contains 21k classes and 14M images [13]. To assess performance at larger scale, we also train on JFT-300M, a proprietary dataset with 300M examples and 18k classes [44]. We de-duplicate all pre-training datasets with respect to the test sets of the downstream tasks as done in Dosovitskiy et al. [14], Kolesnikov et al. [22]. We pre-train all models at resolution 224 using Adam with $\beta_1 = 0.9$, $\beta_2 = 0.999$, linear learning rate warmup of 10k steps and linear decay, batch size 4 096, weight decay, and gradient clipping at global norm 1. For JFT-300M, we pre-process images by applying the cropping technique from Szegedy et al. [45] in addition to random horizontal flipping. For ImageNet and ImageNet-21k, we employ additional data augmentation and regularization techniques. In particular, we use RandAugment [12], mixup [60],